

Kafka Consumers

**Understand how to read data
from Kafka**

Objectives

- Kafka Consumer Concepts
- Partition Rebalance
- Creating a Kafka Consumer
- Configuring Consumer
- Commit and Offset
- Rebalance Listeners
- Clean Exit

Kafka Consumer Concepts

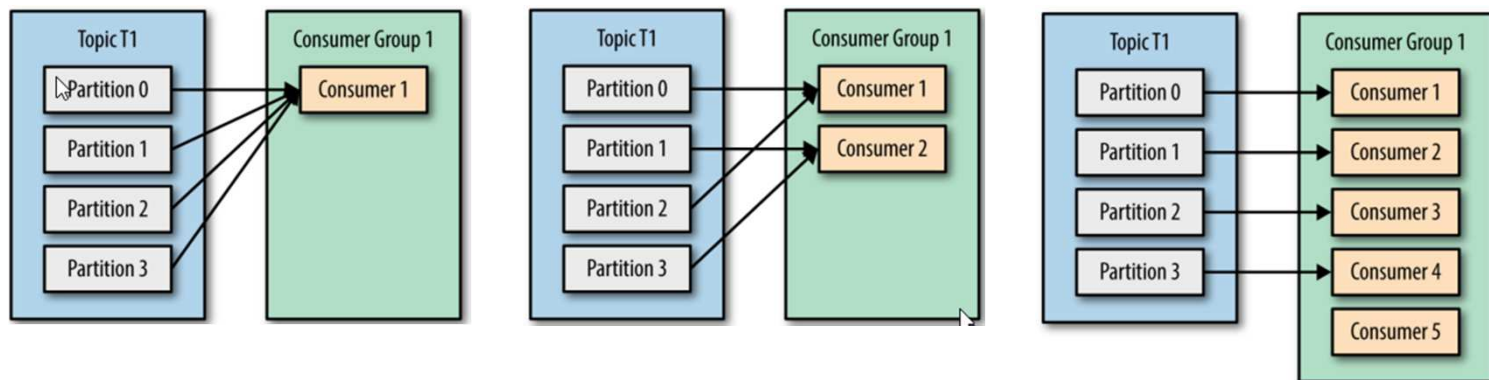
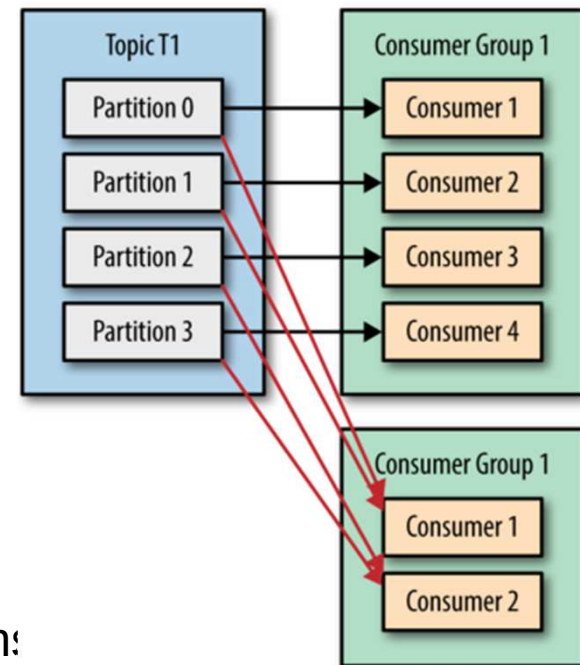
- Consumers and Consumer Groups

- Consumer

- Create a consumer object
 - Subscribe to the appropriate topic
 - Start receiving messages
 - Validate them and write the results

- Consumer Groups

- Kafka consumers are typically part of a consumer group
 - Scaling - Add more consumers to a single group with a single topic having multiple partitions, so that each consumer get a sub set of messages



Consumer Groups and Partition Rebalance

- Consumers in a consumer group share ownership of the partitions in the topics they subscribe to.
- Moving partition ownership from one consumer to another is called a **rebalance**.
- During a rebalance, consumers can't consume messages
- When partitions are moved from one consumer to another, the consumer loses its current state; if it was caching any data, it will need to refresh its caches slowing down the application until the consumer sets up its state again.

Consumers membership with consumer group

- The way consumers maintain membership in a consumer group and ownership of the partitions assigned to them is by sending heartbeats to a Kafka broker designated as the group coordinator.
- Heartbeats are sent when the consumer polls (i.e., retrieves records) and when it commits records it has consumed.
- If the consumer stops sending heartbeats for long enough, its session will time out and the group coordinator will consider it dead and trigger a rebalance.
- When closing a consumer cleanly, the consumer will notify the group coordinator that it is leaving, and the group coordinator will trigger a rebalance immediately, reducing the gap in processing.
- `max.poll.interval.ms` – If `poll()` is not called before expiration of this timeout, then the consumer is considered failed and the group will rebalance in order to reassign the partitions to another member.

How to write Consumer code

- A Kafka producer has four (first 3 mandatory) properties:
 - `bootstrap.servers` – List of `host:port` pairs of brokers that the producer will use to establish initial connection to the Kafka cluster.
 - `key.serializer` – Name of a class that will be used to serialize keys of records
 - `value.serializer` – Name of class that is used to serialize values of records
 - `group.id` – specifies the consumer group the `KafkaConsumer` instance belongs to
- Instantiate Consumer instance
- Subscribing to Topics
 - Method takes a list of topics as a parameter
 - It is also possible to subscribe with a regular expression.
- The Poll Loop
 - Start with an infinite loop, as consumers are usually long-running applications that continuously poll Kafka for more data (`while(true){...}`)
 - Consumer should keep polling else it is considered dead (`consumer.poll(100);`)
 - Iterate through `ConsumerRecords`, where in each item `ConsumerRecord` contains topic, partition, offset, key, value
 - Close the consumer when done (using `finally {consumer.close();}`)

Sample Program

```
Properties kafkaProps = new Properties();
kafkaProps.put("bootstrap.servers", "localhost:2181");
kafkaProps.put("group.id", "invalid");
kafkaProps.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
kafkaProps.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
```

```
KafkaConsumer<String, String> consumer = new KafkaConsumer<String, String>(kafkaProps);
consumer.subscribe(Collections.singletonList("invalid"));
```

```
try {
    int i=0;
    System.out.println("----- Started Polling For Data ----- ");
    while (i < 200) {
        i++;
        ConsumerRecords<String, String> records = consumer.poll(100);
        for (ConsumerRecord<String, String> record : records){
            System.out.printf("Partition = %d, offset = %d, key = %s, value = %s\n",
                record.partition(), record.offset(), record.key(), record.value());
        }
    }
    System.out.println("----- Completed Polling ----- ");
    consumer.close();
}
```

Practice - ConsumerDemo.java

- Open new command prompt, and connect to VM using 'vagrant ssh'
- Set Classpath
 - `export CLASSPATH=./usr/local/kafka/libs/kafka-clients-0.9.0.1.jar:/usr/local/kafka/libs/slf4j-api-1.7.6.jar:/usr/local/kafka/libs/slf4j-log4j12-1.7.6.jar:/usr/local/kafka/libs/log4j-1.2.17.jar`
- Write the code
 - Take the code from 'OneWayConsumerDemo - ReferenceCode'
Note: there are mistakes in ReferenceCode, fix them before proceeding
 - Save code to `c:\labs\kafka\src` folder as "ConsumerDemo.java"
- Compile
 - `cd /vagrant/src`
 - `javac ConsumerDemo.java`
- Run Consumer (this programs waits for 100x200 ms, if it terminates run again)
 - `java ConsumerDemo`
- Using a new command prompt execute ProducerDemo program and generate messages.

Practice - Configuring Loggers

- Create a new properties file in `c:/labs/kafka/src` folder by name `log4j.properties` holding contents from 'log4j reference' file
- Run the `ConsumerDemo` program to notice logs folder getting created with log files in it.

Configuring Consumers

<http://kafka.apache.org/documentation.html#newconsumerconfigs>

- Most of the parameters have reasonable defaults and do not require modification.
- Some of the more important properties.
 - `fetch.min.bytes` – Minimum amount of data a consumer wants to receive, if new records amount is fewer in bytes then broker waits for more messages before sending it to consumer.
 - `fetch.max.wait.ms` – (500 ms), tells Kafka how long to wait if messages are fewer in bytes defined by *fetch.min.bytes*
 - `max.partition.fetch.bytes` – maximum number of bytes the server will return per partition.
 - `session.timeout.ms` – (3 seconds) amount of time a consumer can be out of contact with the brokers while still considered alive
 - `auto.offset.reset` – What to do when there is no initial offset in Kafka (earliest, latest, none – exception to consumer, anything else – exception to consumer)
 - `enable.auto.commit` – If true the consumer's offset will be periodically committed in the background.

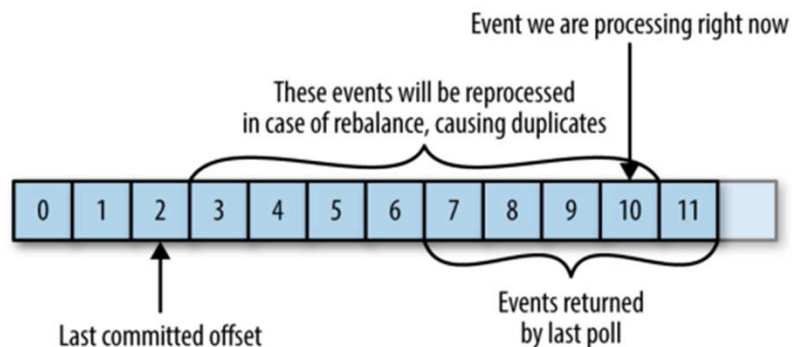
Configuring Consumers

<http://kafka.apache.org/documentation.html#newconsumerconfigs>

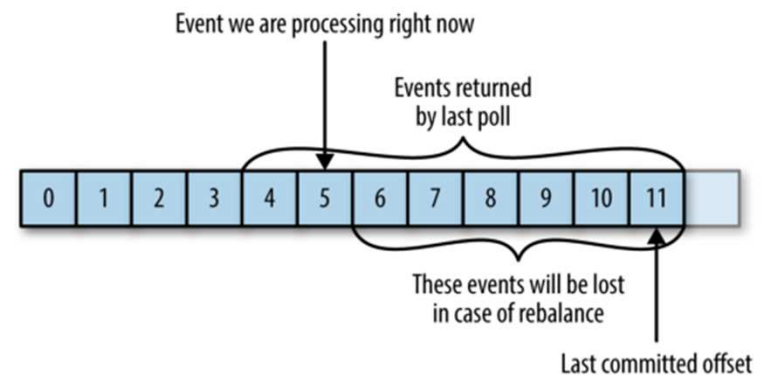
- `partition.assignment.strategy` – The class name for partition assignment algorithm.
 - `org.apache.kafka.clients.consumer.RangeAssignor` (Default - suppose there are two consumers C0 and C1, two topics t0 and t1, and each topic has 3 partitions, resulting in partitions t0p0, t0p1, t0p2, t1p0, t1p1, and t1p2. The assignment will be: C0: [t0p0, t0p1, t1p0, t1p1] C1: [t0p2, t1p2])
 - `org.apache.kafka.clients.consumer.RoundRobinAssignor` (suppose there are two consumers C0 and C1, two topics t0 and t1, and each topic has 3 partitions, resulting in partitions t0p0, t0p1, t0p2, t1p0, t1p1, and t1p2. The assignment will be: C0: [t0p0, t0p2, t1p1] C1: [t0p1, t1p0, t1p2])
- `client.id` – The purpose of this is to be able to track the source of requests beyond just ip/port by allowing a logical application name to be included in server-side request logging.
- `max.poll.records` – The maximum number of records returned in a single call to `poll()`.
- `receive.buffer.bytes` and `send.buffer.bytes` – The size of the TCP receive and send buffer (`SO_RCVBUF` & `SO_SNDBUF`) to use when reading data. When -1 then OS default will be used.
 - `SO_RCVBUF` int Specifies the total per-socket buffer space reserved for receives.
 - `SO_SNDBUF` int Specifies the total per-socket buffer space reserved for sends.

Commits and Offsets

- One of Kafka's unique characteristics is that it does not track acknowledgments from consumers the way many JMS queues do.
- It allows consumers to use Kafka to track their position (offset) in each partition.
- Action of updating the current position in the partition a **commit**
- Produces a message to Kafka, to a special `__consumer_offsets` topic, with the committed offset for each partition.
- If a consumer crashes or a new consumer joins the consumer group, this will trigger a **rebalance**. In order to know where to pick up the work, the consumer will read the latest committed offset of each partition and continue from there.



Committed offset is smaller than last message offset
Result: Messages are reprocessed twice.



Committed message offset is larger than current message offset. Result: messages are lost.

Commit and Offsets ways

- Automatic Commit
 - Easiest way is let Kafka manage it “enable.auto.commit=true”
 - auto.commit.interval.ms (5 seconds) should also to be considered which is the frequency in milliseconds that the consumer offsets are auto-committed to Kafka if enable.auto.commit is set to true.
 - With autocommit enabled, a call to poll will always commit the last offset returned by the previous poll. It doesn't know which events were actually processed, so it is critical to always process all the events returned by poll() before calling poll() again.
- Commit Current Offset
 - Enabled using auto.commit.offset=false
 - Achieved by “consumer.commitSync();” method call in consumer
- Asynchronous Commit
 - Enabled using auto.commit.offset=false
 - One drawback of manual commit is that the application is blocked until the broker responds to the commit request.
 - Achieved by “consumer.commitAsync();” method call.

Practice - Do it yourself

- Modify ConsumerDemo.java to add below property
`kafkaProps.put("enable.auto.commit", "false");`
- Compile and Run ConsumerDemo multiple times.
- What behavior do you notice?
- How to fix this, by keeping `enable.auto.commit` as false?
- Apply the fix in ConsumerDemo.java, compile and run again.

Consuming Records with Specific Offsets

- Use `poll()` to start consuming messages from the last committed offset in each partition and to proceed in processing all messages in sequence.
- Sometimes we want to start reading at a different offset.
 - Start reading all messages from the beginning of the partition
 - `seekToBeginning(TopicPartition tp)`
 - Skip all the way to the end of the partition and start consuming only new messages
 - `seekToEnd(TopicPartition tp)`

Rebalance Listeners

- If consumer need to do some cleanup work before exiting and also before partition rebalancing.
- Implement 'ConsumerRebalanceListener' interface and override below methods
 - onPartitionsAssigned – A callback method the user can implement to provide handling of customized offsets on completion of a successful partition re-assignment.
 - onPartitionsRevoed – A callback method the user can implement to provide handling of offset commits to a customized store on the start of a rebalance operation.

How to cleanly exit

- Ensure `consumer.close()` method is called

Summary

- Kafka Consumer Concepts
- Partition Rebalance
- Creating a Kafka Consumer
- Configuring Consumer
- Commit and Offset
- Rebalance Listeners
- Clean Exit

Practice

- This practice teaches